

Updated: Dec 1

Time Series II

Learning from the Past to Predicting the Future

Approaches II (without lag data in the input sequence)

<https://www.linkedin.com/pulse/data-windowing-technique-used-time-series-forecasting-alejandro/>

Sequence-to-One future value prediction (Seq2One)

- Use **several past time steps** to predict **one future value**.

If we have a time series of temperature, humidity, and pressure:

Time	Temp	Humidity	Pressure
t-3	...	...	...
t-2	...	...	...
t-1	...	...	...
t	?	?	?

Then the model uses the data at times **t-3, t-2, t-1** to predict the **temperature at time t**.

Sequence-to-one (Seq2one) forecasting

Let

$$X_t = [x_{t-3}, x_{t-2}, x_{t-1}]$$

be the past data,
and

$$y_t = x_t$$

be the next value to predict.

The LSTM learns a function

$$\hat{y}_t = f_{\theta}(X_t)$$

that outputs the predicted next value.

We train the model so the predicted value $\hat{y}_t$ is close to the true value y_t :

$$\text{Minimize } (y_t - \hat{y}_t)^2$$

Seq to one, Univariate example

```

----- Original data:
[0.04967142 0.08600699 0.26343818 0.44782319 0.366003 0.45601184
0.72256375 0.72096116 0.67040865 0.83758291]
----- X_train data:
[[[0.04967142]
  [0.08600699]
  [0.26343818]]

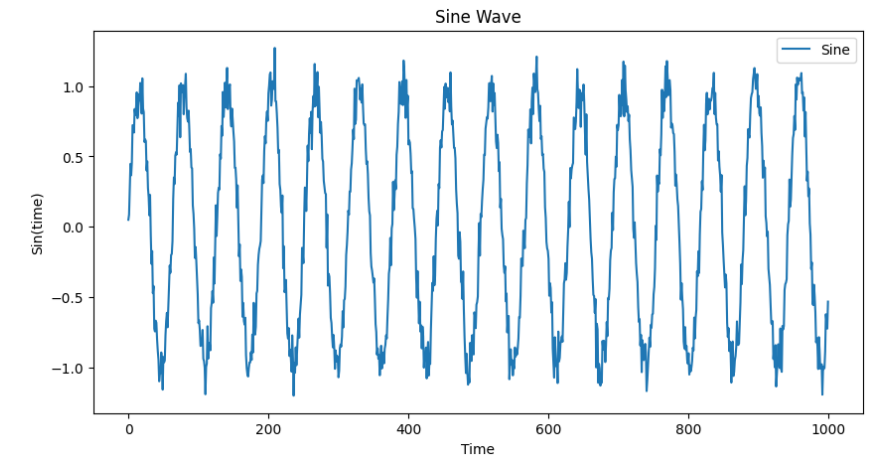
 [[0.08600699]
  [0.26343818]
  [0.44782319]]

 [[0.26343818]
  [0.44782319]
  [0.366003  ]]

 [[0.44782319]
  [0.366003  ]
  [0.45601184]]

 [[0.366003  ]
  [0.45601184]
  [0.72256375]]]
----- y_train data:
[0.44782319 0.366003 0.45601184 0.72256375 0.72096116]

```



TimeSeries_Univari_HPO.ipynb

Seq to one, multivariate example

TimeSeries_seq2one Multivariate.ipynb

Data shape: (300, 3)

	temperature	humidity	pressure
0	20.248357	59.171005	1018.378494
1	19.980868	60.438153	1017.513938
2	20.423844	62.733987	1018.335136
3	20.911515	63.565572	1018.454501
4	20.082923	63.873282	1017.812022
5	20.132932	64.911583	1018.326311
6	21.089606	66.924090	1016.739783
7	20.733717	65.850605	1016.201884
8	20.165263	67.720658	1015.594173
9	20.721280	67.631076	1016.856072
10	20.268291	68.197029	1016.028694
11	20.317135	70.010850	1015.240188
12	20.720981	70.145807	1014.951773
13	19.693360	70.449091	1013.774750
14	19.837541	71.159976	1015.072712
15	20.468856	69.995954	1013.418297
16	20.293584	70.677689	1012.908700
17	21.007124	69.606381	1012.718661
18	20.445988	70.062643	1012.104494
19	20.243848	69.332858	1011.495494

→
scaled

-----scaled data:

[0.03554696 0.46523833 0.96642258]
[0.01841457 0.51734616 0.89165302]
[0.04678672 0.61175573 0.96267281]
[0.07802149 0.64595229 0.97299592]
[0.02495111 0.65860596 0.9174323]
[0.02815408 0.70130311 0.96190958]
[0.08942805 0.78406167 0.82470167]
[0.06663375 0.73991772 0.77818238]
[0.03022486 0.81681827 0.72562568]
[0.06583715 0.81313447 0.83475867]
[0.03682371 0.83640763 0.76320443]
[0.03995212 0.91099584 0.69501192]
[0.06581801 0.91654555 0.67006886]
[0.00923465 0.92901725 0.56827613]
[0.00923465 0.95825034 0.68052801]
[0.04966969 0.91038326 0.53744894]
[0.03844372 0.93841766 0.49337739]
[0.08414512 0.89436321 0.47694222]
[0.048205 0.91312565 0.42382713]
[0.03525817 0.88311534 0.37115888]

```
--- X shape: (297, 3, 3)
```

```
[[[0.03554696 0.46523833 0.96642258]
  [0.01841457 0.51734616 0.89165302]
  [0.04678672 0.61175573 0.96267281]]]
```

```
[[[0.01841457 0.51734616 0.89165302]
  [0.04678672 0.61175573 0.96267281]
  [0.07802149 0.64595229 0.97299592]]]
```

```
[[[0.04678672 0.61175573 0.96267281]
  [0.07802149 0.64595229 0.97299592]
  [0.02495111 0.65860596 0.9174323 ]]]]
```

```
[[[0.07802149 0.64595229 0.97299592]
  [0.02495111 0.65860596 0.9174323 ]
  [0.02815408 0.70130311 0.96190958]]]
```

```
[[[0.02495111 0.65860596 0.9174323 ]
  [0.02815408 0.70130311 0.96190958]
  [0.08942805 0.78406167 0.82470167]]]]]
```

```
--- y shape: (297,)
```

```
[0.07802149 0.02495111 0.02815408 0.08942805 0.06663375 0.03022486
 0.06583715 0.03682371 0.03995212 0.06581801]
```

```
-----scaled data:
```

```
[[[0.03554696 0.46523833 0.96642258]
  [0.01841457 0.51734616 0.89165302]
  [0.04678672 0.61175573 0.96267281]]]
```

```
[0.07802149 0.64595229 0.97299592]
[0.02495111 0.65860596 0.9174323 ]
```

```
[0.02815408 0.70130311 0.96190958]
[0.08942805 0.78406167 0.82470167]
```

```
[0.06663375 0.73991772 0.77818238]
[0.03022486 0.81681827 0.72562568]
```

```
[0.06583715 0.81313447 0.83475867]
[0.03682371 0.83640763 0.76320443]
```

```
[0.03995212 0.91099584 0.69501192]
[0.06581801 0.91654555 0.67006886]
```

```
[0.          0.92901725 0.56827613]
[0.00923465 0.95825034 0.68052801]
```

```
[0.04966969 0.91038326 0.53744894]
[0.03844372 0.93841766 0.49337739]
```

```
[0.08414512 0.89436321 0.47694222]
[0.048205    0.91312565 0.42382713]
```

```
[0.03525817 0.88311534 0.37115888]]]
```

Kera return_sequences

- return_sequences parameter in recurrent layers (like LSTM, GRU, or SimpleRNN) determines whether the layer returns the output for every time step in the input sequence or only the final output at the last time step
- When return_sequences=False (default): This is typically used when the recurrent layer is the last layer in the network or is followed by a standard Dense layer
- When return_sequences=True: **Stacking Recurrent Layers.**

Example:

```
model = Sequential([
    LSTM(64, input_shape=(X_train.shape[1], X_train.shape[2]), return_sequences=True),
    LSTM(32, activation='relu'),
    Dense(32, activation='relu'),
    Dense(1)
])
```

If you have temperature readings like this:

Time	Temp
t-5	...
t-4	...
t-3	...
t-2	...
t-1	...
t	?
t+1	?
t+2	?

Then:

- Input sequence (X) = [t-5, t-4, t-3, t-2, t-1]
- Output sequence (Y) = [t, t+1, t+2]

Seq to Seq forecasting

Use several past time steps to predict several future time steps, not just the next one, but a *sequence* of upcoming values.

The model predicts multiple future steps at once.

Let

$$X_t = [x_{t-L}, x_{t-L+1}, \dots, x_{t-1}]$$

be the past sequence (length L),
and

$$Y_t = [x_t, x_{t+1}, \dots, x_{t+H-1}]$$

be the future sequence (length H).

Then the model learns:

$$\hat{Y}_t = f_{\theta}(X_t)$$

where f_{θ} is the LSTM (or encoder-decoder) model that maps an **input sequence** to an **output sequence**.

Training goal:

Minimize the average error across all predicted steps:

$$\text{Loss} = \frac{1}{H} \sum_{i=0}^{H-1} (x_{t+i} - \hat{x}_{t+i})^2$$

Seq2Seq forecasting

Seq2Seq X, y Example (Univariate)

```
# to test create_sequences(series, look_back, n_future):
test_series = np.arange(12) # 0 ... 11
print(test_series)
X, y = create_sequences(test_series, look_back=5, n_future=3)
print(f"---- X\n{X}")
print(f"---- y\n{y}")
print(f"---- y[:,0]\n{y[:, 0]}")
```

A minor bug in this function

```
[ 0  1  2  3  4  5  6  7  8  9 10 11]
---- X
[[0 1 2 3 4]
 [1 2 3 4 5]
 [2 3 4 5 6]
 [3 4 5 6 7]]
---- y
[[ 5  6  7]
 [ 6  7  8]
 [ 7  8  9]
 [ 8  9 10]]
---- y[:,0]
[5 6 7 8]
```

3. Prepare dataset for sequence-to-sequence forecasting. See below for testing

```
def create_sequences(series, look_back=20, n_future=5):
    X, y = [], []

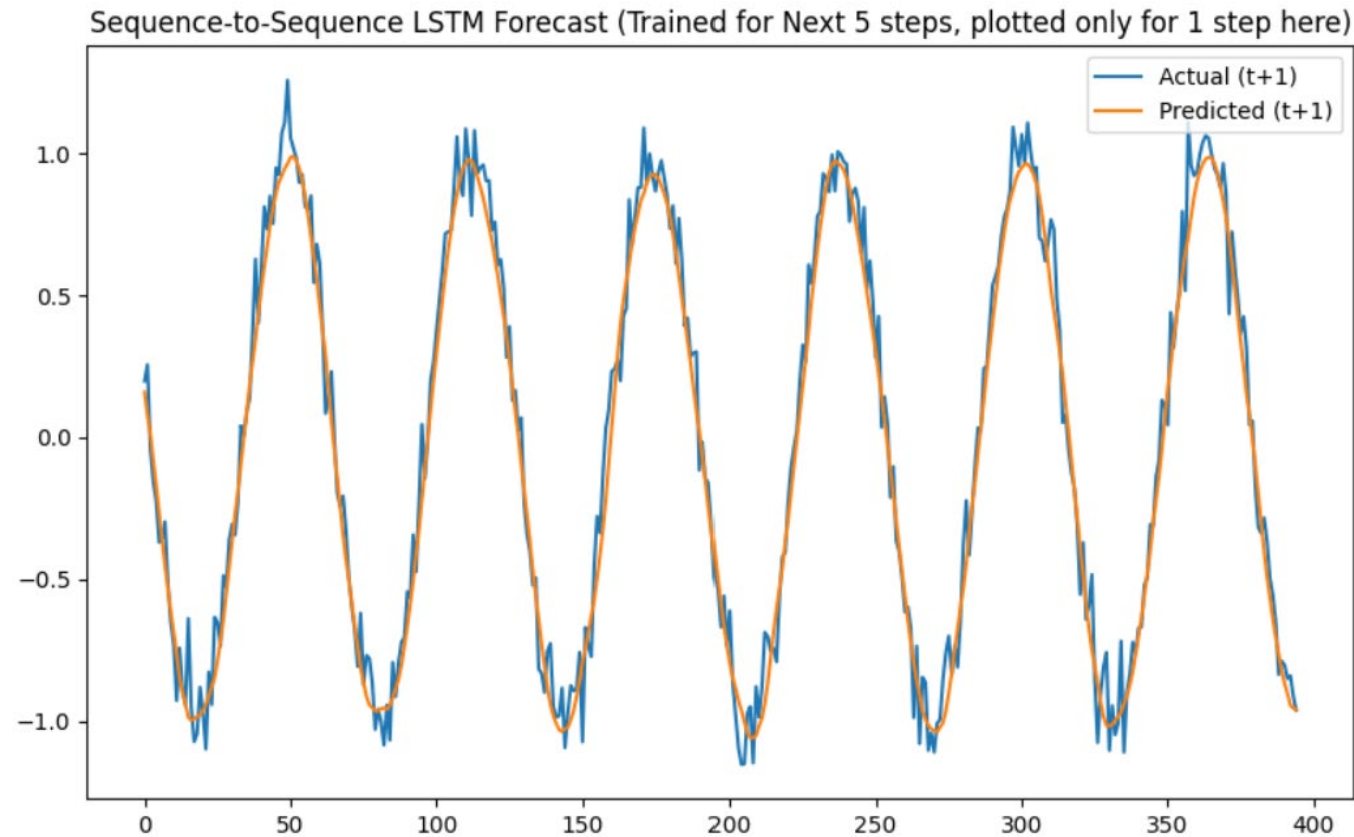
    for i in range(len(series) - look_back - n_future + 1): # Modified: +1 to include the last element
        X.append(series[i : i+look_back])
        y.append(series[i+look_back : i+look_back+n_future])
    return np.array(X), np.array(y)
```

look_back=5, n_future=3

```
[ 0  1  2  3  4  5  6  7  8  9 10 11]
---- X
[[0 1 2 3 4]
 [1 2 3 4 5]
 [2 3 4 5 6]
 [3 4 5 6 7]]
---- y
[[ 5  6  7]
 [ 6  7  8]
 [ 7  8  9]
 [ 8  9 10]]
---- y[:,0]
[5 6 7 8]
```

```
[ 0  1  2  3  4  5  6  7  8  9 10 11]
---- X
[[0 1 2 3 4]
 [1 2 3 4 5]
 [2 3 4 5 6]
 [3 4 5 6 7]
 [4 5 6 7 8]]
---- y
[[ 5  6  7]
 [ 6  7  8]
 [ 7  8  9]
 [ 8  9 10]
 [ 9 10 11]]
---- y[:,0]
[5 6 7 8 9]
```

TimeSeries_seq2seq_univariate.ipynb – Sin wave forecasting

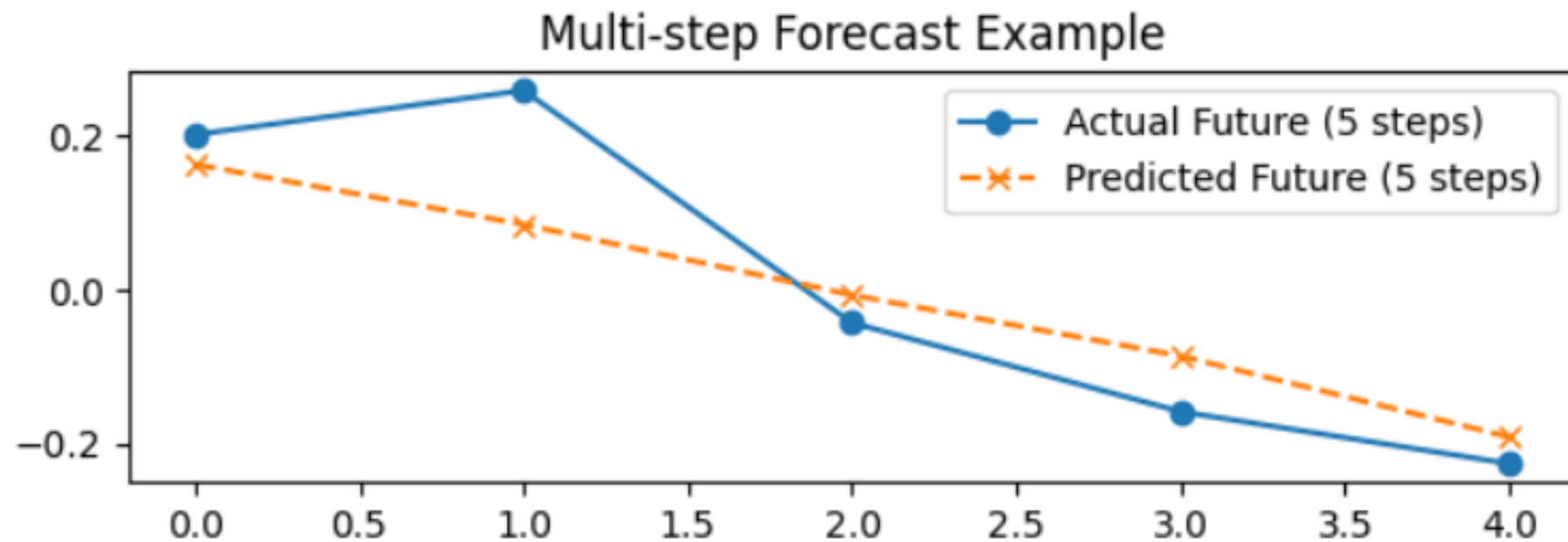


```
plt.figure(figsize=(10, 6))
plt.plot(y_test_inverted[:, 0], label='Actual (t+1)')
plt.plot(pred_inverted[:, 0], label='Predicted (t+1)')
plt.title('Sequence-to-Sequence LSTM Forecast (Trained for Next 5 steps, plotted only for 1 step here)')
plt.legend()
plt.show()
```

Sample index: 0

Actual Future (5 steps): [0.20113561 0.25841628 -0.04337268 -0.15861242 -0.22609826]

Predicted Future (5 steps): [0.16239549 0.08439195 -0.00722752 -0.08642635 -0.19158167]

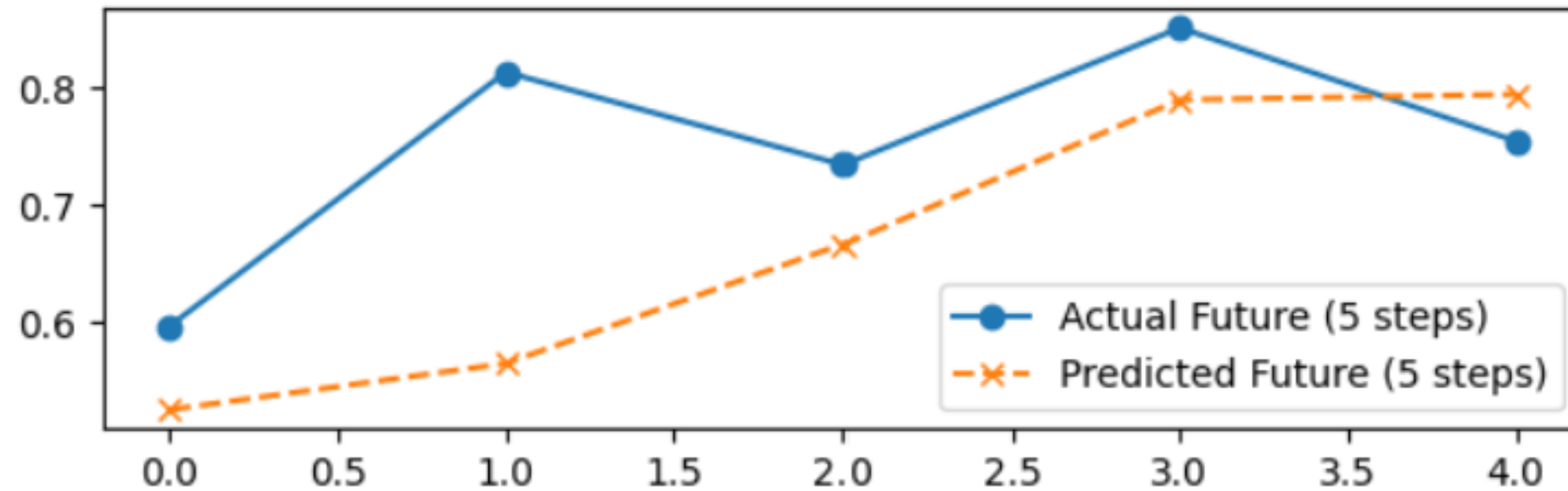


Sample index: 40

Actual Future (5 steps): [0.59735472 0.8134263 0.73479513 0.85113006 0.75473493]

Predicted Future (5 steps): [0.5260277 0.5654435 0.6667205 0.7898504 0.7945497]

Multi-step Forecast Example



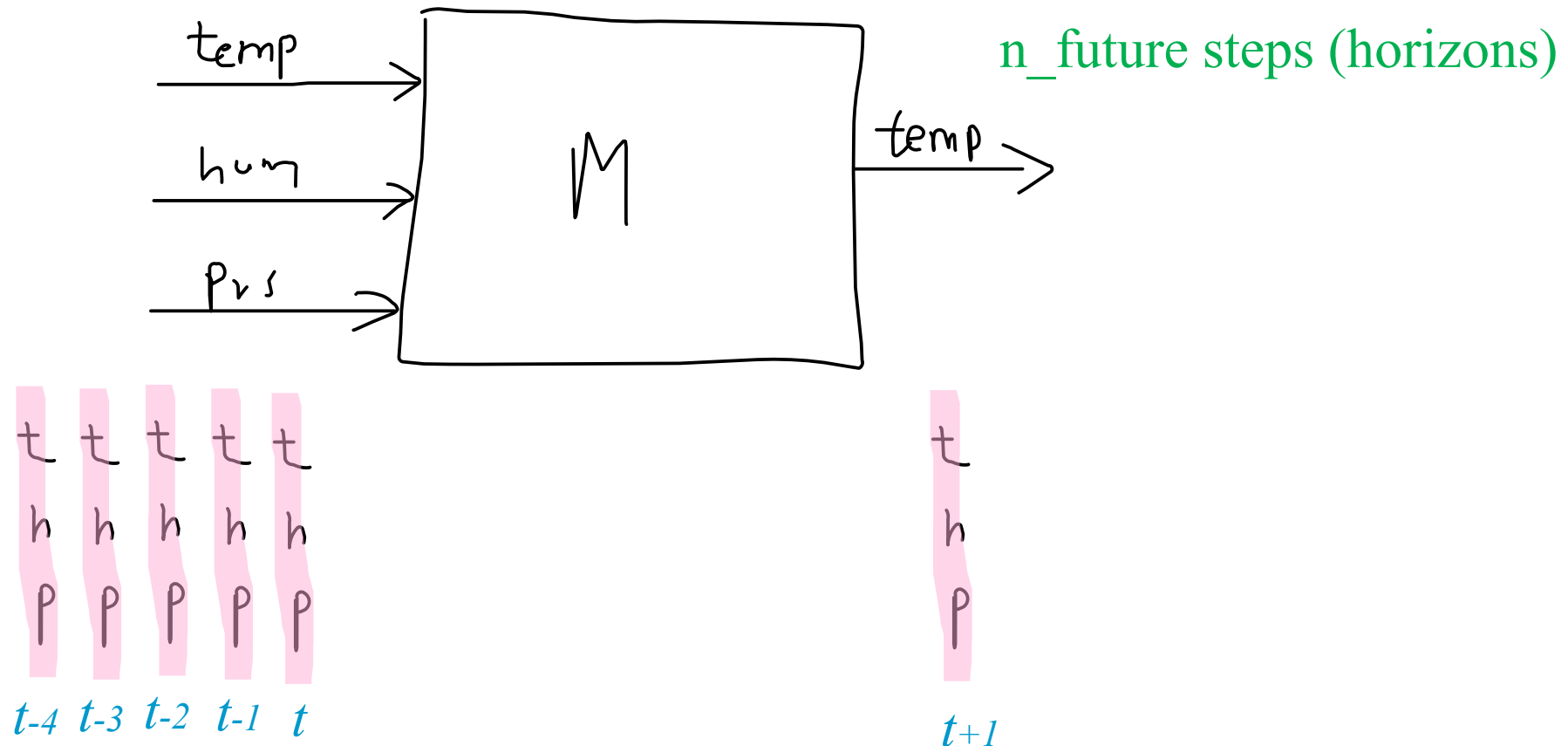
Suggested Project 5: sequence-to-sequence (seq2seq) time series prediction using “Jena Weather Dataset”

- Seq2Seq time series prediction means using past time-series data (a sequence) to predict future time-series data (another sequence) — not just one next point, but several future steps.
- Multi-step horizons (Not just one step ahead, “seq2one”)
- Optimize the model using 2 EAs
- <https://www.kaggle.com/datasets/harishedison/jena-weather-dataset/data>
- Jena Weather dataset is made up of **14 different quantities** (such as air temperature, atmospheric pressure, humidity, wind direction, and so on) that were recorded every 10 minutes, over several years. This dataset covers data from January 1st, 2010 to August 31st, 2022.
- **Due date for all extra credit work: Friday, Dec 12. Note: Final exam: Dec 17 at 8pm**

~~12~~ 19

TimeSeries_seq2seq_multivariate.ipynb (single output)

m_lookback steps



```
# input windows (look_back) and future output sequences (n_future)
def create_seq2seq_dataset(series, look_back=20, n_future=5):
    X, y = [], []
    for i in range(len(series) - look_back - n_future + 1): # added + 1 to include last elt
        X.append(series[i : i + look_back, :]) # all 3 features
        y.append(series[i + look_back : i + look_back + n_future, 0]) # predict Temperature only
    return np.array(X), np.array(y)
```

```
test_series = np.arange(36).reshape(12, 3) # 12 arrays with 3 variables each
print(test_series)
X, y = create_seq2seq_dataset(test_series, look_back=5, n_future=3)
print(f"---- X\n{X}")
print(f"---- y\n{y}")
```

test_series

```
[[ 0  1  2]
 [ 3  4  5]
 [ 6  7  8]
 [ 9 10 11]
 [12 13 14]
 [15 16 17]
 [18 19 20]
 [21 22 23]
 [24 25 26]
 [27 28 29]
 [30 31 32]
 [33 34 35]]
```

```
---- X
[[[ 0  1  2]
 [ 3  4  5]
 [ 6  7  8]
 [ 9 10 11]
 [12 13 14]]
```

```
[[ 3  4  5]
 [ 6  7  8]
 [ 9 10 11]
 [12 13 14]
 [15 16 17]]
```

```
[[ 6  7  8]
 [ 9 10 11]
 [12 13 14]
 [15 16 17]
 [18 19 20]]
```

```
[[ 9 10 11]
 [12 13 14]
 [15 16 17]
 [18 19 20]
 [21 22 23]]
```

```
[[12 13 14]
 [15 16 17]
 [18 19 20]
 [21 22 23]
 [24 25 26]]]
```

```
---- y
[[15 18 21]
 [18 21 24]
 [21 24 27]
 [24 27 30]
 [27 30 33]]
```

Check out
TimeSeries_seq2seq_multivariate.ipynb
on Canvas